



**Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza**

**BASES DE DATOS
CURSO 2024-2025**

Memoria-Segunda Base de Datos

Grupo Lunes 8:00-10:00

L1.02

Autores:

Juan Ondiviela Pamplona: 897735@unizar.es

Eduardo Sánchez Sarsa: 901813@unizar.es

Rubén Villar Artajona: 896654@unizar.es



**Universidad
Zaragoza**



Universidad Zaragoza

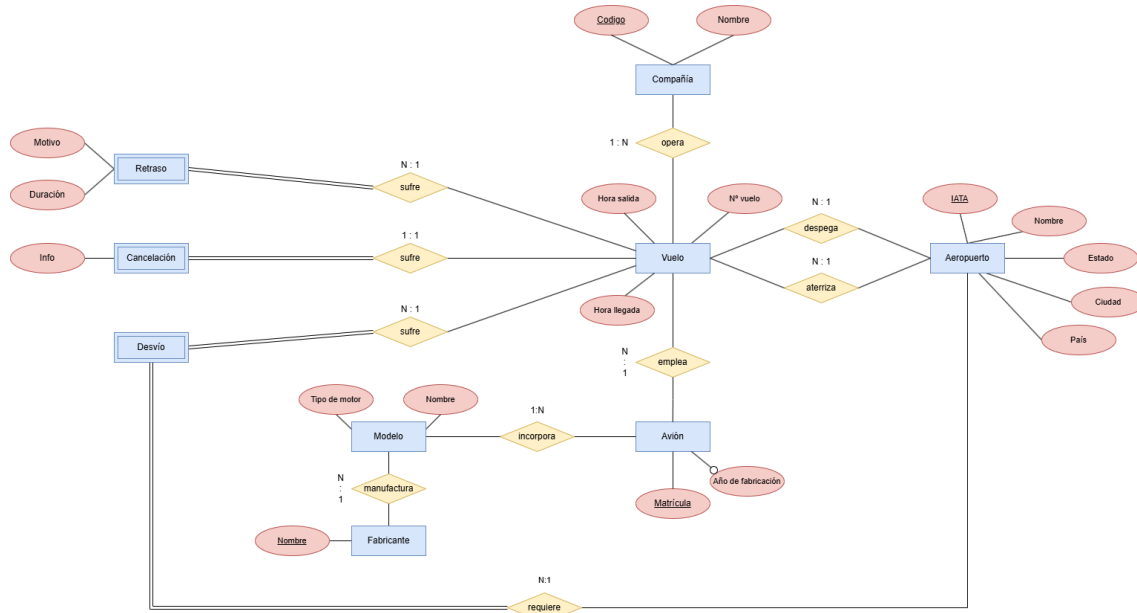
Índice

Práctica 2 - Base de datos de aviones	2
1. Creación de una base de datos	2
1.1. Esquema Entidad-Relación	2
1.2. Restricciones de las entidades	3
1.3. Esquema Relacional	4
1.4. Normalización	5
1.5. Sentencias SQL de creación de tablas	5
2. Introducción de datos y ejecución de consultas	8
2.1. Población de la base de datos	8
2.2. Consultas SQL	9
3. Diseño Físico	14
3.1. Restricciones de Oracle	14
3.2. Estudio de la mejora de rendimiento de las consultas	14
3.3. Realización de triggers	20
CONCLUSIÓN	24
ANEXO I - Registro de horas dedicadas al trabajo	25
ANEXO II - Árboles de álgebra de las consultas	26
ANEXO III - Planes de las consultas	28

Práctica 2 - Base de datos de aviones

1. Creación de una base de datos

1.1. Esquema Entidad-Relación



Este esquema Entidad-Relación (ER) representa un sistema de gestión de vuelos aéreos. Las entidades principales son Aeropuerto, Compañía, Fabricante, Modelo, Avión y Vuelo, con entidades débiles para Cancelación, Desvío y Retraso que dependen de Vuelo.

Las relaciones clave son:

- Un Aeropuerto (identificado por IATA) puede ser origen o destino de muchos Vuelos.
- Una Compañía (con código único) opera múltiples Vuelos.
- Un Fabricante produce varios Modelos de avión.
- Cada Modelo puede ser usado por muchos Aviones.
- Cada Avión (identificado por Matrícula) realiza múltiples Vuelos.

Las entidades débiles muestran eventos especiales:

- Cancelación (1:1 con Vuelo)
- Desvío (un Vuelo puede tener varios Desvíos identificados por ID compuesto)
- Retraso (un Vuelo puede tener múltiples registros de Retraso por diferentes motivos).

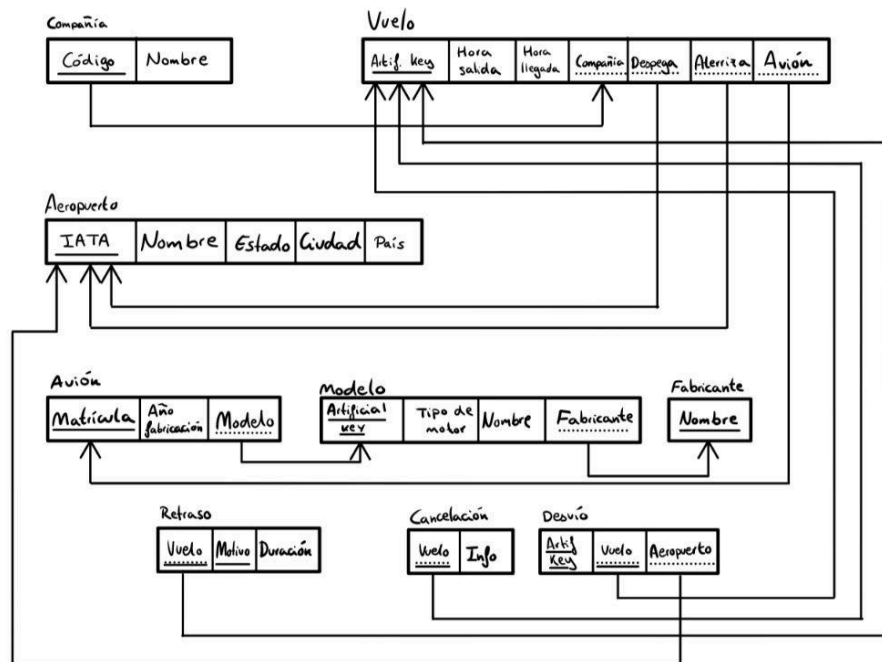
Las restricciones y claves foráneas definen las dependencias obligatorias entre entidades, como que todo Vuelo debe tener aeropuerto de salida/destino, compañía operadora, y puede opcionalmente tener avión asignado. El modelo sigue el estándar de la aviación estadounidense (país fijo como USA, matrículas que comienzan con N) y mantiene integridad referencial mediante las claves primarias y foráneas.

1.2. Restricciones de las entidades

Las restricciones de las entidades son las siguientes:

- Vuelo:
 - Un vuelo puede sufrir más de un retraso.
 - Un vuelo solo puede sufrir una cancelación.
 - Un vuelo no puede ser operado por varias compañías.
 - Un vuelo siempre tiene que tener aeropuerto de salida y destino.
- Compañía:
 - La clave primaria de la compañía va a ser su código.
 - 2 compañías no pueden tener el mismo nombre.
- Retraso:
 - La clave primaria de retraso está compuesta por la ID de vuelo y el motivo del retraso
 - Un retraso tiene que tener motivo.
 - Duración tiene que ser >0 (distinto de NULL)
- Cancelación:
 - Debe de tener información de la causa de la cancelación.
- Desvío:
 - Debe de tener un aeropuerto al que es desviado.
- Aeropuerto:
 - La clave primaria es el IATA, identificador del aeropuerto.
 - El aeropuerto debe de tener nombre, estado y ciudad.
- Avión:
 - Un avión tiene como clave primaria la matrícula.
 - Pero tanto el año de fabricación es opcional, al igual que el modelo.
 - Un avión puede realizar distintos vuelos, a distintos destinos.
- Modelo:
 - El modelo tiene como atributos obligatorios tanto el tipo de motor como el nombre.
- Fabricante:
 - Tiene como clave primaria un fabricante.

1.3. Esquema Relacional



Se han decidido usar varias claves artificiales para varias entidades:

La primera de ellas en la entidad “Vuelo”. El motivo principal para esta decisión es la propagación de claves y la complicada clave primaria compuesta por varios atributos que se quedaría de no usar una clave artificial. Puesto que se propaga hasta en 3 relaciones, sería costoso e ineficiente usar una clave primaria compuesta de varios atributos.

La segunda clave artificial que se ha usado ha sido en la entidad “Modelo”. Esta decisión fue tomada debido a motivos de redundancia que se darían a la hora de propagar las claves hacia la entidad Avión debido a la relación 1:N entre las mismas.

La tercera clave artificial que hemos utilizado ha sido en la entidad de “Desvío”, decisión que se toma debido a la posibilidad de la ocurrencia de varios desvíos para el mismo vuelo, suceso que debido a la falta de atributos para la debida diferenciación de dichas ocurrencias, vemos más fácil la creación de dicha clave.

1.4. Normalización

- **Primera forma normal:** La primera forma normal se cumple, ya que en el modelo E-R original no existen atributos multivaluados.
- **Segunda forma normal:** La segunda forma normal se cumple debido al cumplimiento de la primera forma normal y a la ausencia de dependencias funcionales con parte de las claves.
- **Tercera forma normal:** La tercera forma normal se cumple debido al cumplimiento de la segunda forma normal y a la ausencia de dependencias funcionales transitivas.
- **Forma normal de Boyce-Codd:** La forma normal de Boyce-Codd se cumple debido al cumplimiento de la tercera forma normal y al hecho de que todas las dependencias funcionales forman parte de la clave.

1.5. Sentencias SQL de creación de tablas

```
CREATE TABLE Aeropuerto (  
    -- La clave primaria es el IATA, el resto de atributos son  
obligatorios.  
    -- El país en nuestro caso siempre va a ser 'USA'.  
    IATA VARCHAR(3) PRIMARY KEY,  
    Nombre VARCHAR(50) NOT NULL,  
    Estado VARCHAR(5) NOT NULL,  
    País VARCHAR(15) NOT NULL,  
    Ciudad VARCHAR(40) NOT NULL  
);  
  
CREATE TABLE Compagnia (  
    -- La clave primaria de la compañía será su código.  
    -- Restricción explicada en memoria.  
    Codigo VARCHAR(10) PRIMARY KEY,  
    Nombre VARCHAR(100)  
);  
  
CREATE TABLE Fabricante (  
    -- El atributo único Fabricante es clave primaria.  
    Fabricante VARCHAR(30) PRIMARY KEY  
);  
  
CREATE TABLE Modelo (  

```

```
-- Se usa una clave artificial que será clave primaria.
-- El nombre, motor y Fabricante son atributos obligatorios.
ID VARCHAR (4) PRIMARY KEY,
Nombre VARCHAR(20) NOT NULL,
Motor VARCHAR(20) NOT NULL,
Fabricante VARCHAR(30) NOT NULL,
FOREIGN KEY (Fabricante) REFERENCES Fabricante(Fabricante)
);

CREATE TABLE Avion (
    Matricula VARCHAR(6) PRIMARY KEY,
    ID_Modelo VARCHAR(30),           -- PUEDE SER NULL
    Agno VARCHAR(4),               -- PUEDE SER NULL
    FOREIGN KEY (ID_Modelo) REFERENCES Modelo(ID)
);

CREATE TABLE Vuelo (
    -- Como se puede observar, ID, que es la clave artificial es la
    -- clave primaria de la entidad.
    ID VARCHAR (5) PRIMARY KEY,
    -- El numero de vuelo no puede ser nulo.
    Numero VARCHAR(4) NOT NULL,
    -- La fecha del vuelo no puede ser nula.
    Fecha DATE NOT NULL,
    -- Al igual que la matrícula, el aeropuerto de destino y llegada y
    -- la compañía que opera el vuelo.
    HoraSalida NUMBER,
    HoraSalidaEsperada NUMBER,
    HoraLlegada NUMBER,
    HoraLlegadaEstimada NUMBER,
    Matricula VARCHAR(10) NOT NULL,
    IATAdest VARCHAR(3) NOT NULL,
    IATAsali VARCHAR(3) NOT NULL,
    Company VARCHAR(10) NOT NULL,
    FOREIGN KEY (Matricula) REFERENCES Avion(Matricula),
    FOREIGN KEY (IATAdest) REFERENCES Aeropuerto(IATA),
    FOREIGN KEY (IATAsali) REFERENCES Aeropuerto(IATA),
    FOREIGN KEY (Company) REFERENCES Compagnia(Codigo)
);
```

```
CREATE TABLE Cancelacion (  
    -- La clave primaria es la ID del vuelo, y Causa es un atributo  
obligatorio.  
    ID_Vuelo VARCHAR(8),  
    Causa VARCHAR(20) NOT NULL,  
    PRIMARY KEY (ID_Vuelo),  
    FOREIGN KEY (ID_Vuelo) REFERENCES Vuelo(ID)  
);  
  
CREATE TABLE Desvio (  
    -- La clave primaria es la clave Artificial de desvio y la clave  
primaria de Vuelo.  
    -- La clave del aeropuerto de desvío no puede ser nulo.  
    ID VARCHAR (5),  
    ID_Vuelo VARCHAR(8),  
    IATA_desv VARCHAR(3) NOT NULL,  
    PRIMARY KEY (ID,ID_Vuelo),  
    FOREIGN KEY (ID_VUELO) REFERENCES Vuelo(ID),  
    FOREIGN KEY (IATA_desv) REFERENCES Aeropuerto(IATA)  
);  
  
CREATE TABLE Retraso (  
    -- La clave primaria es la ID del vuelo y el motivo del retraso.  
    -- La duración no puede ser ni NULO ni > 0  
    ID_Vuelo VARCHAR(8),  
    Motivo VARCHAR(15),  
    Duracion NUMBER NOT NULL,  
    PRIMARY KEY (ID_Vuelo, Motivo),  
    FOREIGN KEY (ID_Vuelo) REFERENCES Vuelo(ID)  
    CHECK (Duracion > 0)  
);
```

2. Introducción de datos y ejecución de consultas

2.1. Población de la base de datos

Para la población de esta base de datos, se ha decidido utilizar, al igual que la base de datos anterior, el lenguaje de programación Python, esta vez de forma algo diferente. Se han diseñado 2 scripts en Python, el primero de ellos se ha usado para crear los diferentes ficheros .CSV en los cuales se tendrá el contenido relevante para cada entidad/relación, de manera que, en caso de ser necesaria una modificación, se pueda hacer de manera rápida y eficiente.

El segundo script lo que hace es coger la información de esos ficheros .CSV y crear ficheros .SQL con sentencias de inserción sobre las tablas de la base de datos.

Esta compartimentalización se ha hecho con el fin de poder hacer cambios de manera eficiente y para solucionar rápidamente los problemas a que se encuentran al poblar estas bases de datos.

Problemas como por ejemplo la posibilidad de que un atributo sea nulo que en un principio no se pensaba como nulo, cosa que ha sucedido a la hora de poblar los aviones, (en la base de datos a veces solo aparecen con el atributo matrícula).

Otro problema que se ha encontrado durante la población de la base de datos ha sido la creación de las claves artificiales, debido a que se usan varias de ellas, se han debido de tratar adecuadamente en el script de Python, las restricciones de clave única han resultado abundantes a lo largo del proceso de población de la base.

Quitando estos problemas, y gracias a la experiencia adquirida en el desarrollo de la base de datos anterior, poblar esta base de datos ha dado bastantes menos problemas y ha ocupado menos tiempo que la anterior.

También cabe destacar que el script de Python no se conecta directamente a la base de datos a poblar ya que eso dió muchos problemas en la práctica anterior, por lo que la creación de estos ficheros .SQL ha resultado ser más sencillo.

2.2. Consultas SQL

CONSULTA 1

```
-- Para este primer alias lo que vamos a hacer
-- es calcular el retraso promedio de los
-- vuelos de cada compañía, con el fin
-- de luego coger la compañía necesaria.
WITH RETRASOS_COMPAGNIAS AS (
    SELECT
        V.Company AS Codigo_Compania,
        AVG(R.Duracion) AS Retraso_Promedio
    FROM Vuelo V
    JOIN Retraso R ON V.ID = R.ID_Vuelo
    WHERE R.Duracion IS NOT NULL
    GROUP BY V.Company
),
-- Este alias lo que va a calcular son las compañías
-- que operan al menos 1000 vuelos los días de la base
-- de datos (que son 3 en este caso).
COMPANIAS_VUELOS_MINIMOS AS (
    SELECT
        V.Company AS Codigo_Compania,
        V.Fecha,
        COUNT(V.ID) AS Total_Vuelos
    FROM Vuelo V
    GROUP BY V.Company, V.Fecha
    HAVING COUNT(V.ID) >= 1000
),
-- Este alias lo que hará será coger las compañías
-- que cumplan los 1000 vuelos los 3 días de datos
COMPANIAS_CUMPLEN_REQUISITO_DIAS AS (
    SELECT Codigo_Compania
    FROM COMPANIAS_VUELOS_MINIMOS
    GROUP BY Codigo_Compania
    HAVING COUNT(*) = 3
)
-- Por último, se cogerán los datos de los promedios
-- de retraso antes calculados con un JOIN a las
-- compañías que cumplen los requisitos
```

```

SELECT C.NOMBRE AS NOMBRE, RC.RETRASO_PROMEDIO
FROM RETRASOS_COMPAGNIAS RC, COMPANIAS_CUMPLEN_REQUISITO_DIAS CR,
COMPAGNIA C
WHERE RC.CODIGO_COMPANIA = CR.CODIGO_COMPANIA AND C.CODIGO =
CR.CODIGO_COMPANIA AND C.CODIGO = RC.CODIGO_COMPANIA;

```

Álgebra de la consulta.

Resultado:

NOMBRE	MEDIA_RETRASO
Southwest Airlines Co.	5,59899385
Skywest Airlines Inc.	9,49521707
American Airlines Inc.	12,1709123
American Eagle Airlines Inc.	12,4055444

CONSULTA 2

```

-- Este alias calcula la compañía que tiene opera más aviones.
-- Seleccionando el que más matrículas distintas de aviones
-- opera en sus vuelos.
WITH COMPAGNIA_MAX_AVIONES AS (
    SELECT C.Codigo AS CODE, C.Nombre
    FROM Compagnia C
    JOIN Vuelo V ON C.Codigo = V.Company
    GROUP BY C.Codigo, C.Nombre
    HAVING COUNT(DISTINCT V.Matricula) = (
        SELECT MAX(NumAviones)
        FROM (
            SELECT C2.Codigo, COUNT(DISTINCT V2.Matricula) AS NumAviones
            FROM Compagnia C2
            JOIN Vuelo V2 ON C2.Codigo = V2.Company
            GROUP BY C2.Codigo
        )
    )
),
-- Este alias lo que hace es seleccionar todos los aeropuertos
-- en los que opera la compañía que más aviones tiene.
AEROPUERTOS_COMPAGNIAS AS (

```

```

SELECT DISTINCT V.IATAdest AS dest, V.IATAsali AS sali
FROM Vuelo V
WHERE V.Company IN (SELECT CODE FROM COMPAGNIA_MAX_AVIONES)
)
-- Y por último, seleccionamos solo los aeropuertos que no están
-- en el alias anterior en Alaska y California.
SELECT A.IATA, A.Nombre, A.Estado
FROM Aeropuerto A
WHERE A.IATA NOT IN (SELECT dest FROM AEROPUERTOS_COMPAGNIAS) AND A.IATA
NOT IN (SELECT sali FROM AEROPUERTOS_COMPAGNIAS) AND A.Estado IN ('AK',
'CA');

```

Álgebra de la consulta.

Resultado:

IAT NOMBRE	ESTAD
CEC Jack McNamara	CA
ADK Adak	AK
MOD Modesto City-County-Harry Sham	CA
SBA Santa Barbara Municipal	CA
OME Nome	AK
PMD Palmdale Production Flight	CA
BET Bethel	AK
SIT Sitka	AK
MRY Monterey Peninsula	CA
RDD Redding Municipal	CA
ADQ Kodiak	AK
OXR Oxnard	CA
JNU Juneau International	AK
CDV Merle K (Mudhole) Smith	AK
YAK Yakutat	AK
KTN Ketchikan International	AK
SBP San Luis Obispo Co-McChesney	CA
BRW Wiley Post Will Rogers Memorial	AK
ACV Arcata	CA
OAK Metropolitan Oakland International	CA
BFL Meadows	CA
WRG Wrangell	AK
LGB Long Beach (Daugherty)	CA
FAI Fairbanks International	AK
CIC Chico Municipal	CA
SCC Deadhorse	AK
SMX Santa Maria Pub/Capt G Allan Hancock	CA
ANC Ted Stevens Anchorage International	AK

OTZ Ralph Wien Memorial	AK
PSG James C. Johnson Petersburg	AK
IYK Inyokern	CA
IPL Imperial County	CA

CONSULTA 3

```
-- Este alias lo que hace es asociar los aviones que pasan por
-- cada aeropuerto y asociar Aeropuerto - Edad del avión
WITH AVIONES_AEROPUERTOS AS (
SELECT DISTINCT A.nombre AS NOMBRE, A.iata AS IATA, (2025 - AV.agno) as
edad, AV.MATRICULA
FROM aeropuerto A, vuelo V, avion AV
WHERE (A.IATA = V.IATAsali or A.IATA = V.IATAdest) AND AV.Matricula =
V.Matricula AND AV.AGNO IS NOT NULL
),
-- Este alias lo que hace es calcular el promedio de la edad
-- de cada aeropuerto agrupando.
PROMEDIO_EDAD AS (
    SELECT AA.NOMBRE, AA.IATA, AVG(edad) AS PROMEDIO
    FROM AVIONES_AEROPUERTOS AA
    GROUP BY AA.NOMBRE,AA.IATA
)
-- Por último, se coge de las medias calculadas
-- la menor de todas, mediante ese WHERE PE.promedio
-- que tiene un SELECT dentro, de manera que si varios
-- tuviesen la misma media se cogiesen todos.
SELECT *
FROM PROMEDIO_EDAD PE
WHERE PE.PROMEDIO = (
    SELECT MIN(PROMEDIO) as minimo
    FROM PROMEDIO_EDAD
);
```

Álgebra de la consulta.

Resultado:

NOMBRE	IAT	PROMEDIO
-----	----	-----
Yampa Valley	HDN	20,8888889

3. Diseño Físico

3.1. Restricciones de Oracle

- El país del aeropuerto siempre ha de ser USA.
- El año de fabricación válido deberá estar contenido en el intervalo [1900-año_actual].
- Del mismo modo que la anterior restricción, la fecha de los vuelos no puede ser posterior a la actual.
- La duración en Retraso ha de ser positiva.
- Debe de haber coherencia en las fechas de las posibles incidencias para el mismo vuelo (Desvío/Cancelación/Retraso). También han de ser posteriores a la del vuelo.
- El código del estado del aeropuerto debe ser válido.
- La duración del vuelo debe ser mayor que 0 en caso de que no esté cancelado.
- Debe haber coherencia entre las horas de salida-llegada ($h. salida < h. llegada$).
- Un vuelo no puede tener Retraso si está en Cancelación.

3.2. Estudio de la mejora de rendimiento de las consultas

Mejora de rendimiento de la consulta 1

```
-- Este alias lo que va a calcular son las compañías
-- que operan al menos 1000 vuelos los días de la base
-- de datos (que son 3 en este caso).
CREATE INDEX codigo_compagnia_idx ON vuelo(Company);
CREATE MATERIALIZED VIEW COMPANIAS_VUELOS_MINIMOS AS
  SELECT
    V.Company AS Codigo_Compania,
    V.Fecha,
    COUNT(V.ID) AS Total_Vuelos
  FROM Vuelo V
  GROUP BY V.Company, V.Fecha
  HAVING COUNT(V.ID) >= 1000;
-- Para este primer alias lo que vamos a hacer
-- es calcular el retraso promedio de los
-- vuelos de cada compañía, con el fin
-- de luego coger la compañía necesaria.
WITH RETRASOS_COMPAGNIAS AS (
  SELECT
    V.Company AS Codigo_Compania,
```

```

        AVG(R.Duracion) AS Retraso_Promedio
    FROM Vuelo V
    JOIN Retraso R ON V.ID = R.ID_Vuelo
    WHERE R.Duracion IS NOT NULL
    GROUP BY V.Company
),
-- Este alias lo que hará será coger las compañías
-- que cumplan los 1000 vuelos los 3 días de datos
COMPANIAS_CUMPLEN_REQUISITO_DIAS AS (
    SELECT Codigo_Compania
    FROM COMPANIAS_VUELOS_MINIMOS
    GROUP BY Codigo_Compania
    HAVING COUNT(*) = 3
)
-- Por último, se cogerán los datos de los promedios
-- de retraso antes calculados con un JOIN a las
-- compañías que cumplen los requisitos
SELECT C.NOMBRE AS NOMBRE, RC.RETRASO_PROMEDIO
FROM RETRASOS_COMPAGNIAS RC, COMPANIAS_CUMPLEN_REQUISITO_DIAS CR,
COMPAGNIA C
WHERE RC.CODIGO_COMPANIA = CR.CODIGO_COMPANIA AND C.CODIGO =
CR.CODIGO_COMPANIA AND C.CODIGO = RC.CODIGO_COMPANIA;

```

La consulta 1 reformada optimiza el rendimiento respecto a la original mediante dos mejoras:

Crea una vista materializada que precalcula y almacena permanentemente las compañías con más de 1000 vuelos por día, evitando recalcular este dato costoso en cada ejecución

Añade un índice sobre el campo Company para acelerar las operaciones de agrupación y unión.

Estas optimizaciones funcionan porque la vista materializada actúa como caché de resultados intermedios (especialmente útil para el filtro HAVING COUNT >= 1000), mientras que el índice reduce drásticamente el tiempo de búsqueda al organizar los datos por compañía.

La nueva consulta optimizada también simplifica el cálculo de retrasos promedios eliminando una subconsulta intermedia.

Los problemas de rendimiento que se habían encontrado previamente a la optimización es que se realizan varios JOIN sobre la tabla más grande que tiene la base de datos que es VUELO, por ello, el coste de hacer estos JOIN es bastante elevado, realizándose

hasta 3 veces este JOIN sobre la tabla VUELO, por lo que la solución más lógico y directa fue al realización de vistas materializadas, sobre todo para el precálculo de estas consultas intermedias que usaban 1 o varios JOIN sobre vuelo.

Mejora de rendimiento de la consulta 2

```
CREATE INDEX idx_vuelo_company ON vuelo(Company);

CREATE INDEX idx_airport_estado ON aeropuerto(Estado);

WITH COMPAGNIA_MAX_AVIONES AS (

    SELECT C.Codigo AS CODE, C.Nombre

    FROM Compagnia C

    JOIN Vuelo V ON C.Codigo = V.Company

    GROUP BY C.Codigo, C.Nombre

    HAVING COUNT(V.Matricula) = (

        SELECT MAX(NumAviones)

        FROM (

            SELECT C2.Codigo, COUNT(V2.Matricula) AS NumAviones

            FROM Compagnia C2

            JOIN Vuelo V2 ON C2.Codigo = V2.Company

            GROUP BY C2.Codigo

        )

    )

),

AEROPUERTOS_COMPAGNIAS AS (

    SELECT DISTINCT V.IATAdest AS dest, V.IATAsali AS sali

    FROM Vuelo V

    WHERE V.Company IN (SELECT CODE FROM COMPAGNIA_MAX_AVIONES)

)

SELECT A.IATA, A.Nombre, A.Estado

FROM Aeropuerto A
```

```
WHERE A.IATA NOT IN (SELECT dest FROM AEROPUERTOS_COMPAGNIAS) AND  
A.IATA NOT IN (SELECT sali FROM AEROPUERTOS_COMPAGNIAS) AND A.Estado IN  
('AK', 'CA');
```

La consulta 2 reformada, optimiza la original mediante dos índices clave (en Company de Vuelo y Estado de Aeropuerto) y eliminando el DISTINCT en los conteos de matrículas, lo que acelera significativamente las operaciones JOIN y GROUP BY al reducir el costo computacional, manteniendo el mismo resultado final (aeropuertos en Alaska/California donde no opera la compañía con más aviones) pero con mejor rendimiento en bases de datos grandes, ya que los índices permiten búsquedas más rápidas y la eliminación de DISTINCT simplifica los cálculos de agregación sin afectar la lógica del negocio.

Los problemas que se encuentran en esta consulta, tienen que ver también con los JOIN sobre la tabla VUELO, en este caso, el alias COMPAGNIA_MAX_AVIONES tiene claramente un coste elevado, debido a que una subconsulta del HAVING, tiene también una selección con JOIN a la tabla VUELO, a parte del otro JOIN de la tabla VUELO.

Este problema se puede resolver fácilmente con una vista materializada, para el otro alias, AEROPUERTOS_COMPAGNIAS, se plantean 2 opciones para optimizarla, la primera de ellas es la creación de una vista materializada también, y la otra, la creación de un índice sobre el campo de Vuelo(Company) para que el JOIN tenga un coste menor debido a que dicho campo optimizando así la consulta, pero tras la prueba de este índice sobre la base de datos, se ha decidido que era mejor opción el uso de la vista materializada.

Mejora de rendimiento de la consulta 3

```
CREATE INDEX idx_vuelo_iatasali ON Vuelo(IATAsali);
CREATE INDEX idx_vuelo_iatadest ON Vuelo(IATAdest);
CREATE INDEX idx_vuelo_matricula ON Vuelo(Matricula);
WITH edadMediaAeropuertos AS (
  SELECT AE.nombre, AE.iata, AVG(2025 - AV.agno) as media
  FROM aeropuerto AE, vuelo V, avion AV
  WHERE (AE.IATA = V.IATAsali or AE.iata = V.IATAdest) AND AV.matricula =
  V.matricula AND AV.agno IS NOT NULL
  GROUP BY AE.nombre, AE.iata
)
SELECT S.nombre, S.iata, S.media
FROM edadMediaAeropuertos S
WHERE S.media = (
  SELECT MIN(media) as minimo
  FROM edadMediaAeropuertos A
);
```

La consulta 3 reformada optimiza la original mediante la creación de tres índices clave (en IATAsali, IATAdest y Matricula de la tabla Vuelo) que aceleran las operaciones JOIN y las condiciones WHERE al reducir el tiempo de búsqueda, manteniendo la misma lógica y resultados (identificar aeropuertos con aviones más modernos calculando la edad media) pero con mejor rendimiento, ya que permiten accesos más rápidos a los datos involucrados en las relaciones entre tablas.

Los problemas de rendimiento encontrados en esta consulta tienen que ver con el triple JOIN que se produce en el alias AVIONES_AEROPUERTOS, ya que es un triple JOIN con las tablas VUELO y AEROPUERTO que son considerablemente pesadas, por lo que tienen un coste muy elevado, se plantea también el uso de varios índices sobre la tabla VUELO, para indexar los campos de COMPANY y MATRÍCULA, pero se acaba tomando la decisión de la realización de otra vista materializada, debido a que el coste de AVIONES_AEROPUERTO sigue siendo bastante elevado.

En el [Anexo III](#) se pueden encontrar los planes de ejecución de las consultas optimizadas y sin optimizar.

3.3. Realización de triggers

Trigger 1

```
/*
 * TRIGGER: verificar_IATA
 *
 * PROPÓSITO: Validar que los códigos IATA de aeropuertos en los vuelos
 * cumplan con el formato estándar internacional (3 letras mayúsculas).
 *
 * ACCIÓN: Se ejecuta antes de insertar o actualizar un registro en la
 * tabla Vuelo
 */
CREATE OR REPLACE TRIGGER verificar_IATA
BEFORE INSERT OR UPDATE ON Vuelo
FOR EACH ROW
BEGIN
    -- Valida que el código IATA de salida tenga exactamente 3 letras
    mayúsculas
    IF NOT REGEXP_LIKE(:NEW.IATAsali, '^[A-Z]{3}$') THEN
        RAISE_APPLICATION_ERROR(-20003, 'Error: El IATA de salida debe
        tener exactamente 3 letras mayúsculas.');

```
 END IF;

 -- Valida que el código IATA de destino tenga exactamente 3 letras
 mayúsculas
 IF NOT REGEXP_LIKE(:NEW.IATAdest, '^[A-Z]{3}$') THEN
 RAISE_APPLICATION_ERROR(-20004, 'Error: El IATA de destino debe
 tener exactamente 3 letras mayúsculas.');

```
    END IF;
END;
/
```


```


```

El primer trigger, "verificar_IATA", valida que los códigos IATA de aeropuertos en la tabla Vuelo cumplan con el formato estándar de 3 letras mayúsculas, verificando tanto el origen (IATAsali) como el destino (IATAdest) antes de cada inserción o actualización, generando errores específicos si no cumplen.

Trigger 2

```
/*
 * TRIGGER: validar_matricula_vuelo
 *
 * PROPÓSITO: Garantizar que las matrículas de aviones sigan el formato
 estándar
 * utilizado en los registros de la FAA (Administración Federal de
 Aviación de EE.UU.)
 *
 * ACCIÓN: Se ejecuta antes de insertar un registro en la tabla Vuelo
 */
CREATE OR REPLACE TRIGGER validar_matricula_vuelo
BEFORE INSERT ON Vuelo
FOR EACH ROW
BEGIN
    -- Valida que la matrícula cumpla con el formato estándar:
    -- Debe comenzar con N seguido de 1 a 6 caracteres alfanuméricos
    IF NOT REGEXP_LIKE(:NEW.Matricula, '^N[0-9A-Z]{1,6}$') THEN
        RAISE_APPLICATION_ERROR(-20004, 'Error: La matrícula debe seguir
 el formato N seguido de 1-6 caracteres alfanuméricos');
    END IF;
END;
```

El segundo trigger, “validar_matricula_vuelo”, asegura que las matrículas de aviones sigan el formato de la FAA (N seguido de 1-6 caracteres alfanuméricos) al insertar nuevos vuelos, rechazando matrículas inválidas.

Trigger 3

```
/*
 * TRIGGER: validar_desvio
 *
 * PROPÓSITO: Este trigger realiza múltiples validaciones para
 garantizar la
 * integridad de los datos en la tabla Desvio, verificando que el
 aeropuerto
 * de desvío no sea el mismo que el de origen
 */
CREATE OR REPLACE TRIGGER validar_desvio
BEFORE INSERT OR UPDATE ON Desvio
FOR EACH ROW
DECLARE
    v_iata_origen VARCHAR(3);
BEGIN
    SELECT IATAsali INTO v_iata_origen FROM Vuelo WHERE ID =
:NEW.ID_Vuelo;
    IF :NEW.IATA_desv = v_iata_origen THEN
        RAISE_APPLICATION_ERROR(-20007, 'Error: El aeropuerto de desvío
no puede ser el mismo que el de origen');
    END IF;
END;
/
```

El tercer trigger, “validar_desvio”, realiza múltiples validaciones para garantizar la integridad de los desvíos: verifica que el aeropuerto de desvío no sea el mismo que el de origen. Para ello, antes de insertar o actualizar un registro en la tabla Desvio, obtiene el código IATA del aeropuerto de origen asociado al vuelo (IATAsali) desde la tabla Vuelo. Si el código IATA del aeropuerto de desvío (IATA_desv) es igual al de origen, genera un error.

CONCLUSIÓN

La realización de esta práctica ha supuesto una experiencia integral en el ámbito del diseño y gestión de bases de datos, permitiendo aplicar de manera rigurosa los conocimientos teóricos adquiridos a lo largo del curso. A partir del estudio de un caso representativo —la gestión de vuelos comerciales en el territorio de los Estados Unidos— se ha llevado a cabo el diseño conceptual mediante un modelo entidad-relación detallado, en el que se han identificado y documentado cuidadosamente las restricciones funcionales, así como las decisiones de modelado adoptadas.

Posteriormente, se procedió a la transformación del modelo conceptual al esquema relacional, realizando un proceso exhaustivo de normalización hasta alcanzar la Forma Normal de Boyce-Codd, lo que garantiza la eliminación de redundancias y la mejora en la integridad de los datos. La implementación de dicho esquema en el sistema gestor de bases de datos Oracle se efectuó mediante sentencias SQL, acompañada de la carga de datos reales, lo que planteó diversos retos técnicos que fueron resueltos con criterios fundamentados.

La segunda fase del trabajo permitió abordar consultas complejas mediante el uso combinado del álgebra relacional y SQL, profundizando en aspectos relacionados con el análisis de eficiencia a través del plan de ejecución de las consultas. En la fase final, se prestó especial atención al diseño físico y a la implementación de restricciones que no pueden ser garantizadas mediante el modelo relacional estándar, utilizando para ello mecanismos como los triggers.

En suma, esta práctica ha proporcionado una visión completa y aplicada del ciclo de vida de una base de datos, desde su concepción inicial hasta su implementación optimizada. Asimismo, ha fomentado la capacidad de análisis, la toma de decisiones fundamentadas, y el trabajo colaborativo, competencias esenciales en el ámbito profesional de las tecnologías de la información.

ANEXO I - Registro de horas dedicadas al trabajo

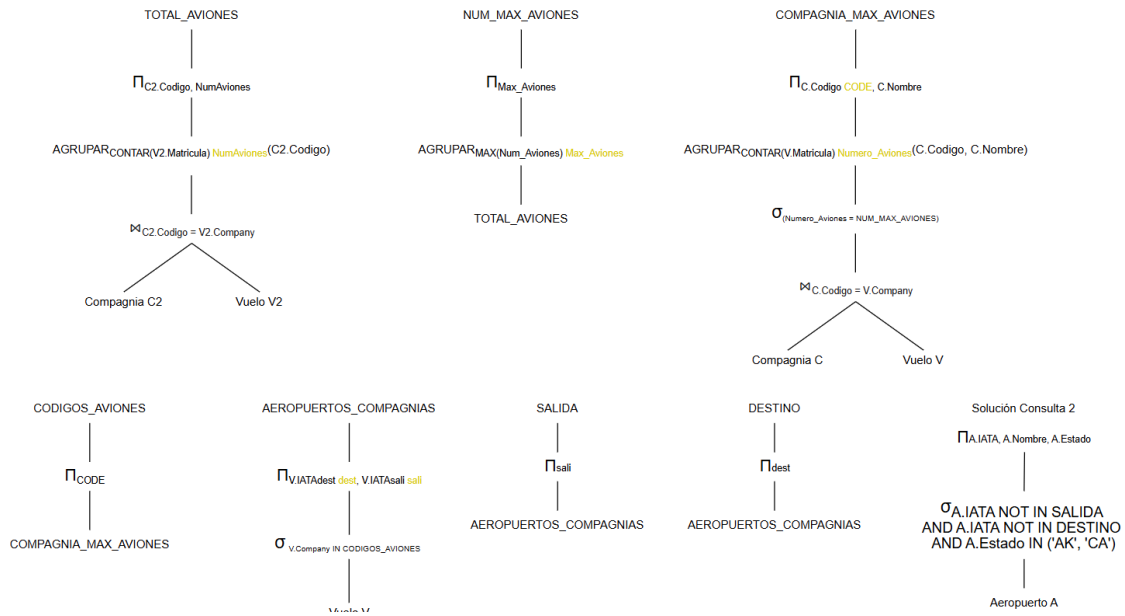
	HORAS DEDICADAS		
	Rubén Villar Artajona	Eduardo Sánchez Sarsa	Juan Ondiviela Pamplona
Trabajo en clase de prácticas	6	6	6
Parte 1. Creación de una base de datos (TOTALES)	9	7	9
Esquema Entidad Relación	3	3	3
Restricciones de las entidades	1	1	1
Esquema Relacional	3	1	1
Normalización	1	1	3
Sentencias SQL de creación de tablas	1	1	1
Parte 2. Introducción de datos y ejecución de consultas (TOTALES)	6	10	8
Población de la base de datos	1	5	1
Consultas SQL	5	5	4
Árboles relacionales	0	0	3
Parte 3. Diseño físico (TOTALES)	4	5	1
Estudio de la mejora del rendimiento de las consultas	1	4	1
Realización de triggers	3	1	0
TOTALES	25	28	24

ANEXO II - Árboles de álgebra de las consultas

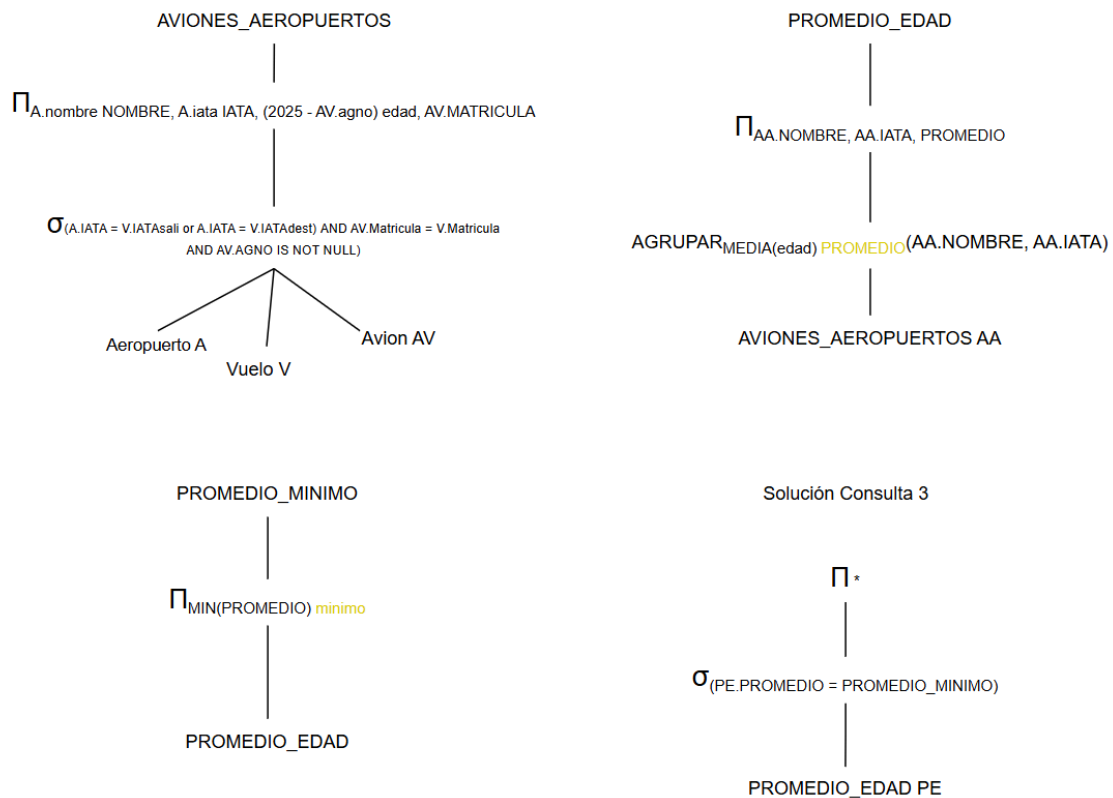
Árbol sintáctico de la consulta 1



Árbol sintáctico de la consulta 2



Árbol sintáctico de la consulta 3



ANEXO III - Planes de las consultas

Plan consulta 1 sin optimizar:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	60	440 (4)	00:00:01
1	SORT ORDER BY		1	60	440 (4)	00:00:01
* 2	HASH JOIN		1	60	439 (4)	00:00:01
3	JOIN FILTER CREATE	:BF0000	1	44	284 (4)	00:00:01
* 4	HASH JOIN		1	44	284 (4)	00:00:01
5	NESTED LOOPS		1	28	142 (4)	00:00:01
6	NESTED LOOPS		1	28	142 (4)	00:00:01
7	VIEW		1	3	141 (4)	00:00:01
* 8	HASH GROUP BY		1	3	141 (4)	00:00:01
9	VIEW		2	6	141 (4)	00:00:01
* 10	FILTER					
11	HASH GROUP BY		2	22	141 (4)	00:00:01
12	TABLE ACCESS FULL	VUELO	48400	519K	138 (2)	00:00:01
* 13	INDEX UNIQUE SCAN	SYS_C008387	1		0 (0)	00:00:01
14	TABLE ACCESS BY INDEX ROWID	COMPAGNIA	1	25	1 (0)	00:00:01
15	VIEW		18	288	141 (4)	00:00:01
16	HASH GROUP BY		18	54	141 (4)	00:00:01
17	TABLE ACCESS FULL	VUELO	48400	141K	138 (2)	00:00:01
18	VIEW		18	288	155 (3)	00:00:01
19	HASH GROUP BY		18	342	155 (3)	00:00:01
20	JOIN FILTER USE	:BF0000	12466	231K	153 (2)	00:00:01
* 21	HASH JOIN		12466	231K	153 (2)	00:00:01
22	TABLE ACCESS FULL	RETRASO	12466	121K	15 (0)	00:00:01
23	TABLE ACCESS FULL	VUELO	48400	425K	138 (2)	00:00:01

Plan consulta 1 optimizada:

Id	Operation	Name	Rows	Bytes	TempSpc	Cost (%CPU)	Time
0	SELECT STATEMENT		1	57	145 (4)	00:00:01	
* 1	HASH JOIN		1	57	145 (4)	00:00:01	
2	NESTED LOOPS		1	32	142 (4)	00:00:01	
3	NESTED LOOPS		1	32	142 (4)	00:00:01	
4	VIEW		1	7	141 (4)	00:00:01	
* 5	HASH GROUP BY		1	3	141 (4)	00:00:01	
6	VIEW	COMPANIAS_VUELOS_MINIMOS	2	6	141 (4)	00:00:01	
* 7	FILTER						
8	HASH GROUP BY		2	22	141 (4)	00:00:01	
9	TABLE ACCESS FULL	VUELO	48400	519K	138 (2)	00:00:01	
* 10	INDEX UNIQUE SCAN	SYS_C008387	1		0 (0)	00:00:01	
11	TABLE ACCESS BY INDEX ROWID	COMPAGNIA	1	25	1 (0)	00:00:01	
12	MAT_VIEW ACCESS FULL	RETRASOS_COMPAGNIAS	18	450	3 (0)	00:00:01	

Plan consulta 2 sin optimizar:

Id	Operation	Name	Rows	Bytes	TempSpc	Cost (%CPU)	Time
0	SELECT STATEMENT		1	32	952 (2)	00:00:01	
* 1	FILTER						
* 2	HASH JOIN ANTI		1	32	478 (2)	00:00:01	
* 3	TABLE ACCESS FULL	AEROPUERTO	10	290	3 (0)	00:00:01	
4	VIEW	VW_NSO_3	4014	12042	474 (2)	00:00:01	
* 5	HASH JOIN		4014	56196	474 (2)	00:00:01	
6	VIEW	VW_NSO_2	1	7	336 (2)	00:00:01	
* 7	FILTER						

8	HASH GROUP BY		1 10	336 (2) 00:00:01
9	VIEW	VM_NWVW_4 48400 472K	336 (2) 00:00:01	
10	HASH GROUP BY		48400 472K 960K	336 (2) 00:00:01
11	TABLE ACCESS FULL	VUELO	48400 472K	138 (2) 00:00:01
12	SORT AGGREGATE		1 13	
13	VIEW		18 234	336 (2) 00:00:01
14	SORT GROUP BY		18 180	336 (2) 00:00:01
15	VIEW	VM_NWVW_5 48400 472K	336 (2) 00:00:01	
16	SORT GROUP BY		48400 472K 960K	336 (2) 00:00:01
17	TABLE ACCESS FULL	VUELO	48400 472K	138 (2) 00:00:01
18	TABLE ACCESS FULL	VUELO	48400 330K	138 (2) 00:00:01
* 19	HASH JOIN SEMI		1 14	474 (2) 00:00:01
20	VIEW	VW_NSO_1 1 7	336 (2) 00:00:01	
* 21	FILTER			
22	SORT GROUP BY		1 10	336 (2) 00:00:01
23	VIEW	VM_NWVW_6 48400 472K	336 (2) 00:00:01	
24	SORT GROUP BY		48400 472K 960K	336 (2) 00:00:01
25	TABLE ACCESS FULL	VUELO	48400 472K	138 (2) 00:00:01
26	SORT AGGREGATE		1 13	
27	VIEW		18 234	336 (2) 00:00:01
28	SORT GROUP BY		18 180	336 (2) 00:00:01
29	VIEW	VM_NWVW_7 48400 472K	336 (2) 00:00:01	
30	SORT GROUP BY		48400 472K 960K	336 (2) 00:00:01
31	TABLE ACCESS FULL	VUELO	48400 472K	138 (2) 00:00:01
* 32	TABLE ACCESS FULL	VUELO	178 1246	138 (2) 00:00:01

Plan consulta 2 optimizada:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	40	2 (0)	00:00:01
1	NESTED LOOPS ANTI		1	40	2 (0)	00:00:01
2	NESTED LOOPS ANTI		1	37	2 (0)	00:00:01
* 3	TABLE ACCESS FULL	AEROPUERTO	42	1428	2 (0)	00:00:01

```
* 4 | INDEX RANGE SCAN | IDX_MV_SALI | 371 | 1113 | 0 (0) | 00:00:01 |
* 5 | INDEX RANGE SCAN | IDX_MV_DEST | 371 | 1113 | 0 (0) | 00:00:01 |
```

Plan consulta 3 sin optimizar:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		524	20436	1208 (1)	00:00:01
* 1	FILTER					
2	HASH GROUP BY		524	20436	1208 (1)	00:00:01
3	VIEW		80346	3060K	1208 (1)	00:00:01
4	HASH UNIQUE		80346	3530K	4448K	1208 (1) 00:00:01
* 5	HASH JOIN		80346	3530K	288 (2)	00:00:01
* 6	TABLE ACCESS FULL	AVION	3435	41220	5 (0)	00:00:01
7	VIEW	VW_JF	96622	3113K	282 (2)	00:00:01
8	UNION-ALL					
* 9	HASH JOIN		48222	1930K	141 (2)	00:00:01
10	TABLE ACCESS FULL	AEROPUERTO	272	7072	3 (0)	00:00:01
11	TABLE ACCESS FULL	VUELO	48400	708K	138 (2)	00:00:01
* 12	HASH JOIN		48400	1748K	141 (2)	00:00:01
13	TABLE ACCESS FULL	AEROPUERTO	272	7072	3 (0)	00:00:01
14	TABLE ACCESS FULL	VUELO	48400	519K	138 (2)	00:00:01
15	SORT AGGREGATE		1	13		
16	VIEW		52315	664K	1208 (1)	00:00:01
17	SORT GROUP BY		52315	1992K	1208 (1)	00:00:01
18	VIEW		80346	3060K	1208 (1)	00:00:01
19	SORT UNIQUE		80346	3530K	4448K	1208 (1) 00:00:01
* 20	HASH JOIN		80346	3530K	288 (2)	00:00:01
* 21	TABLE ACCESS FULL	AVION	3435	41220	5 (0)	00:00:01
22	VIEW	VW_JF_SET\$45E53CEE	96622	3113K	282 (2)	00:00:01
23	UNION-ALL					
* 24	HASH JOIN		48222	1930K	141 (2)	00:00:01
25	TABLE ACCESS FULL	AEROPUERTO	272	7072	3 (0)	00:00:01
26	TABLE ACCESS FULL	VUELO	48400	708K	138 (2)	00:00:01

* 27	HASH JOIN		48400 1748K		141 (2)	00:00:01
28	TABLE ACCESS FULL	AEROPUERTO		272 7072		3 (0) 00:00:01
29	TABLE ACCESS FULL	VUELO		48400 519K		138 (2) 00:00:01

Plan de la consulta 3 optimizada:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		270	8640		52 (6) 00:00:01
* 1	FILTER					
2	HASH GROUP BY		270	8640		52 (6) 00:00:01
3	MAT_VIEW ACCESS FULL	AVIONES_AEROPUERTOS	26951	842K		49 (0) 00:00:01
4	SORT AGGREGATE		1	13		
5	VIEW		26951	342K		288 (2) 00:00:01
6	SORT GROUP BY		26951	842K	1176K	288 (2) 00:00:01
7	MAT_VIEW ACCESS FULL	AVIONES_AEROPUERTOS	26951	842K		49 (0) 00:00:01